Counterfactual Planning Gap

An Interval-Gated Statistic for Diagnosing
World-Model Bottlenecks under a Fixed Planner

Denis Hamon
Independent.
denis.hamon1@gmail.com
Repository: https://github.com/Denis-hamon/world-model-eval-lab

June 14, 2026

Abstract

Action-conditioned world models are usually evaluated by prediction quality (reconstruction loss, frame-level FID, held-out accuracy), which is silent on the question an applied team must answer before integrating a model into a control loop: *does the model, when used by a planner, produce decisions that succeed?* We propose the **Counterfactual Planning Gap (CPG)**: the success-rate difference between a fixed planner using oracle dynamics and the same planner using the learned model, on identical runs that differ only in the `dynamics` callable. We report it with an Agresti–Caffo plus-4 interval (which keeps the variance positive at the boundary proportions $p \in \{0, 1\}$ where the Wald approximation collapses), a paired bootstrap where the design warrants it, and a five-branch verdict (MODEL BOTTLENECK, LEARNED OUTPERFORMS ORACLE, PLANNER BOTTLENECK, MODEL AS GOOD AS ORACLE, INCONCLUSIVE) gated on the lower bound of the CI rather than the point estimate, so under-powered runs cannot over-claim a diagnosis. We package CPG behind a minimal evaluation contract (`wmel`) and exercise it on three DeepMind Control Suite tasks. The central finding is that the verdict is *heterogeneous and condition-sensitive* – precisely what a calibrated metric should surface, and what a point-estimate leaderboard cannot. On Acrobot-swingup the gap looks large only because the oracle's fixed initial state is an unusually easy swing-up; sampling the task's initial-state distribution collapses the oracle to $\sim$3% success and flips the verdict from MODEL BOTTLENECK to PLANNER BOTTLENECK. On Reacher-easy the verdict is MODEL BOTTLENECK across all arms (CPG from +0.20 to +0.33). On Cartpole-swingup at higher model capacity the larger TD-MPC2 [Hansen et al., 2024] under a Cross-Entropy-Method planner *beats* the oracle planner: LEARNED OUTPERFORMS ORACLE, CPG = -0.27, AC CI $[-0.48, -0.02]$ and paired-bootstrap CI $[-0.50, -0.03]$, both clearing zero. We also present this as *self-correction*: an earlier single-fixed-initial-state evaluation of this same framework reported MODEL BOTTLENECK on Acrobot; the metric's own interval-gated machinery, re-run over the task distribution, overturned that headline. Finally, because the gate is a function of the confidence interval, it doubles as a power-analysis tool: we give the per-arm episode count a comparison needs before its interval clears zero, and show that a plausible leaderboard near-tie (0.94 vs 0.92 at $n = 100$) is statistically indistinguishable from noise.

1 Introduction

The world-model literature has converged on three properties that motivate its existence: *prediction* of future observations conditioned on actions, *planning* by rolling those predictions out, and *transfer* across tasks that share latent structure [Ha and Schmidhuber, 2018, Hafner et al., 2023, Bardes et al., 2024, Bruce et al., 2024a]. Most published evaluations focus on the first: reconstruction loss, frame-level Fréchet Inception Distance, or next-frame prediction loss on held-out trajectories. These metrics are easy to compute and easy to compare across releases, but they are silent on a question that any applied team must answer before integrating a learned world model into a control loop: *does using the model to plan produce decisions that succeed at the latency and compute the deployment will tolerate?*

The gap between prediction quality and decision quality is well documented. Hafner et al. [2019] report success rates on DeepMind Control Suite tasks alongside reconstruction loss because the two metrics disagree.

Yu et al. [2020], Kidambi et al. [2020] explicitly study *model exploitation* – the gap between a planner using a learned model and the same planner using the true environment dynamics – in offline model-based RL. Agarwal et al. [2021] document how rapidly point estimates of return mislead at small sample sizes, and prescribe interval reporting. These are all healthy signals; what is missing is a packaged, reusable, decision-grade scalar that quantifies the gap with an honest confidence interval and a decision rule.

This paper makes three modest contributions:

1. **An interval-gated decision rule for the planning gap.** We define the **Counterfactual Planning Gap (CPG)** – the success-rate difference between a fixed planner using oracle dynamics and the same planner using a learned model, on identical runs differing only in their `dynamics` callable (§4) – and report it with an Agresti–Caffo plus-4 interval, a paired bootstrap where the design is paired, and a five-branch verdict gated on the CI lower bound. The diagnosis is thus only as strong as the evidence: under-powered runs return INCONCLUSIVE rather than over-claim. The underlying quantity is the model-exploitation gap of offline model-based RL [Yu et al., 2020, Kidambi et al., 2020]; the contribution is the packaged, gated, interval-reported *statistic and decision procedure*, not the subtraction.

2. **Heterogeneous verdicts across three tasks.** Exercised on three DeepMind Control Suite tasks (Acrobot-swingup, Cartpole-swingup, Reacher-easy), the gate fires four of its five branches on real data (§5–§5.7): PLANNER BOTTLENECK on Acrobot (the oracle planner itself solves only $\sim 3\%$ of random initial states), MODEL BOTTLENECK on Reacher, LEARNED OUTPERFORMS ORACLE on high-capacity Cartpole under CEM, and INCONCLUSIVE on several moderate-n near-ties. A fixed metric, or a point-estimate leaderboard, reports none of this structure.

3. **The metric as self-correction.** An earlier version of this very framework, evaluated at a single fixed initial state, reported a large MODEL BOTTLENECK gap on Acrobot. Re-running over the task's initial-state distribution – the design the metric's own honesty discipline demands – collapses the oracle and overturns the verdict to PLANNER BOTTLENECK (§5.4). The episode is the strongest evidence for the metric's purpose: a calibrated, interval-gated statistic caught a config-sensitive artifact that a point estimate would have published.

The framework is open source[1], runs CPU-only without a GPU, and has no heavy machine-learning dependency at runtime; PyTorch and `dm-control` are pulled in only for the worked-example adapters. CI verifies the contract on Python 3.11/3.12/3.13.

2 Related Work

Latent-dynamics world models. Ha and Schmidhuber [2018], Hafner et al. [2019], Hafner et al. [2020], and Hafner et al. [2023] establish the latent-dynamics + planner pattern that this framework targets. Schrittwieser et al. [2020] demonstrates the same pattern with a value-based search; Micheli et al. [2023] replaces the recurrent backbone with a transformer; Hansen et al. [2024] pushes the planner side toward stochastic MPC at scale.

Joint-embedding predictive architectures. LeCun [2022] proposed JEPA as a self-supervised path to world models that predicts in latent space without reconstructing pixels. Assran et al. [2023] and Bardes et al. [2024, 2025] report results on representation quality. None of these papers report a planning-success-rate gap as a scalar; the framework here is complementary.

Generative world models. Bruce et al. [2024a,b] demonstrate large-scale generative world models conditioned on action sequences. Evaluation in these works is dominated by perceptual and generative metrics; planning impact is not yet a standard reporting axis.

Model exploitation in offline model-based RL. Yu et al. [2020] and Kidambi et al. [2020] measure the gap between a planner using a learned model and the same planner using the true dynamics. They report return curves; we package the same comparison as a single scalar with a confidence interval and a decision rule.

[1] https://github.com/Denis-hamon/world-model-eval-lab

Value-equivalence and decision-aware model learning. A line of work argues that prediction accuracy is the wrong objective for a model that will be planned with, and that what matters is the model's effect on values and decisions. Grimm et al. [2020] formalise this as the *value equivalence principle* and Grimm et al. [2021] as *proper value equivalence*; massoud Farahmand et al. [2017] and massoud Farahmand [2018] derive value-aware and iterative value-aware model-learning losses, and D'Oro et al. [2020] a gradient-aware variant. These are *learning objectives*: they reshape how the model is trained so that error is weighted by its decision impact. CPG is complementary and empirical: it does not change training, but *measures*, closed-loop and with a calibrated interval, how much a given model's residual error costs a fixed planner – an a-posteriori, planner-fixed probe of (proper) value inequivalence. The dissociation we report (held-out prediction loss drops $\sim$150$\times$ while planning success does not move, §5.4) is precisely the phenomenon this literature predicts.

Evaluation methodology. Henderson et al. [2018] document the irreproducibility of RL evaluation under typical sample sizes. Agarwal et al. [2021] prescribes bootstrap-based interval reporting and Pareto-style aggregate metrics. We adopt the closed-form Agresti–Caffo interval rather than a bootstrap for two reasons specific to this setting: at the boundary proportions $p \in \{0, 1\}$ that recur here (a learned arm at $0/n$), a nonparametric bootstrap of the success rate degenerates to a zero-width interval – the same pathology that motivates avoiding the Wald interval – whereas the plus-4 adjustment keeps the variance positive; and only a closed-form half-width is a function of (n, p_o, p_ℓ) that can be inverted *before* any rollouts, which is what makes the power analysis of §5.8 possible. Wilson [1927] provides the Wilson score interval that is the close cousin of the Agresti–Caffo CI we use; Agresti and Caffo [2000] extends it to the difference of two proportions; Brown et al. [2001] survey the boundary-behaviour failures of the Wald interval that motivate both. Newcombe [1998a] catalogues a family of interval methods for the same problem. The CPG CI is a direct application of Agresti–Caffo to the planning-comparison setting.

Benchmarks. We evaluate on Acrobot-swingup from the DeepMind Control Suite [Tassa et al., 2018, Tunyasuvunakool et al., 2020]. Acrobot has been a reference task for model-based RL since at least Schäfer et al. [2007]. Park et al. [2024] and Liu et al. [2023] are the canonical present-day benchmark suites for goal-conditioned and manipulation evaluation; integrating either is in scope for future work. Wang et al. [2019] benchmark model-based RL algorithms on shared tasks; our axis is orthogonal – a per-(model, planner) statistic rather than an algorithm leaderboard.

Evaluation platforms. Maes et al. [2026] is a concurrent, complementary platform for world-model research that standardises data loading, baseline implementations, planning solvers, and a broad environment suite with controllable factors of variation, reporting success rate as its primary control metric.[2] Such platforms report point-estimate success rates; CPG is the decision-grade statistic that a platform of that kind could compute per model per environment to separate the contribution of the model from that of the planner, with a calibrated interval and a decision rule rather than a bare leaderboard number.

3 The Evaluation Contract and a Decision-Grade Taxonomy

3.1 The four-method contract

Every adapter in the framework implements

$$\begin{aligned} \texttt{encode}&: \mathcal{O} \to \mathcal{Z}, \\ \texttt{rollout}&: \mathcal{Z} \times \mathcal{A}^H \to \mathcal{Z}^{H+1}, \\ \texttt{score}&: \mathcal{Z} \times \mathcal{Z} \to \mathbb{R}, \\ \texttt{plan}&: \mathcal{O} \times \mathcal{O} \times \mathbb{N} \to \mathcal{A}^H. \end{aligned}$$

Here $\mathcal{O}$ is the observation space, $\mathcal{Z}$ the latent space (which may coincide with $\mathcal{O}$ for fully observed envs), $\mathcal{A}$ the action space, and H the planning horizon. The runner queries `plan` on every replanning step and executes

[2] CPG and the `wmel` framework were developed independently and concurrently; the public repository's version history predates and does not derive from that platform. The contribution here is orthogonal: a single statistic and decision rule, not a platform, and it composes with any benchmark runner that can swap the dynamics callable.

one or more of the returned actions in the real environment; the planner is free to use `encode`, `rollout`, and `score` internally however it sees fit. The contract makes no statement about how the model is trained.

For the present paper we use a random-shooting MPC: at each `plan` call, sample N_{cand} action sequences of length H_{plan} uniformly from $\mathcal{A}^{H_{\text{plan}}}$, roll each one out through the dynamics, score the resulting trajectories, and return the best-scoring sequence prefix.

3.2 Decision-grade metrics

A metric is *decision-grade* in this framework iff (i) its units translate directly to a deployment-time cost or capability and (ii) it is computable only from closed-loop runs of the model, not from the model in isolation. Reconstruction loss and FID fail (ii); model-internal alignment scores fail (i). The metrics that survive both criteria, and that we report in every scorecard, are: action success rate, average steps to success, per-call planning latency, compute per decision, perturbation recovery rate, the effective planning horizon, and – the contribution of this paper – the Counterfactual Planning Gap.

4 Counterfactual Planning Gap

4.1 Definition

Fix an environment $\mathcal{E}$, a planner π, a scoring function σ, a number of episodes N, a horizon T and a seed. Let success_rate(D) denote the empirical success rate of π on N episodes of $\mathcal{E}$ when the planner queries dynamics D during its internal rollouts. Let $D^\star$ be the *oracle* dynamics (the true env's transition function) and D_θ be a learned model. The Counterfactual Planning Gap is

$$\text{CPG} \;=\; \text{success\_rate}(D^\star) \;-\; \text{success\_rate}(D_\theta). \tag{1}$$

All free quantities in the right-hand side – env, planner, score, N, T, and the initial-state distribution – are held fixed between the two runs. The only thing that changes is the `dynamics` callable passed to the planner, so CPG attributes the success-rate difference to that callable, holding everything else fixed. It is therefore a property of the (model, planner, distribution) triple, *not* a planner- or distribution-free property of the model alone: as §5.4 shows, the same model can flip the verdict when only the evaluation distribution changes. Read CPG relative to a stated planner and a stated distribution.

4.2 Statistical reporting

Let s_o, s_ℓ be the success counts in the oracle and learned arms and n_o, n_ℓ the corresponding episode counts. The raw point estimate of CPG is the difference of proportions:

$$\hat{\Delta} \;=\; \frac{s_o}{n_o} - \frac{s_\ell}{n_\ell}. \tag{2}$$

The standard Wald 95% CI on $\hat{\Delta}$ has variance $p_o(1-p_o)/n_o + p_\ell(1-p_\ell)/n_\ell$, which collapses to zero whenever *either* arm sits at $p \in \{0, 1\}$. With $n_\ell = 10$ episodes and a learned planner that fails on every episode, this is precisely the regime the framework lands in. A degenerate-variance CI in this regime falsely produces a tight interval and over-claims significance.

We instead use the Agresti–Caffo plus-4 adjustment [Agresti and Caffo, 2000], adding one pseudo-success and one pseudo-failure to each arm:

$$\tilde{p}_o = \frac{s_o + 1}{n_o + 2}, \quad \tilde{p}_\ell = \frac{s_\ell + 1}{n_\ell + 2}, \tag{3}$$

$$\tilde{\Delta} = \tilde{p}_o - \tilde{p}_\ell, \tag{4}$$

$$\text{SE} = \sqrt{\frac{\tilde{p}_o(1-\tilde{p}_o)}{n_o + 2} + \frac{\tilde{p}_\ell(1-\tilde{p}_\ell)}{n_\ell + 2}}, \tag{5}$$

$$\text{CI}_{95\%}(\text{CPG}) = \big[\, \tilde{\Delta} - 1.96\,\text{SE},\;\; \tilde{\Delta} + 1.96\,\text{SE} \,\big]. \tag{6}$$

The framework reports both the raw $\hat{\Delta}$ (what a reader expects to see) and the AC interval (what is statistically defensible). The two coincide for large n.

4.3 Gated verdict

A point estimate without a significance gate over-claims. The framework therefore exposes a five-branch decision rule that consults the AC interval, not the raw $\hat{\Delta}$:

- MODEL BOTTLENECK: $\mathrm{CI}_{\mathrm{lo}} > 0$. The oracle is reliably better; closing the gap is a model problem.
- LEARNED OUTPERFORMS ORACLE: $\mathrm{CI}_{\mathrm{hi}} < 0$. Rare; investigate regularisation or planner-search interactions.
- PLANNER BOTTLENECK: CI crosses 0 *and* both success rates are within a tolerance τ of 0. Neither planner solves the task; the framework needs a stronger search, not a stronger model.
- MODEL AS GOOD AS ORACLE: CI crosses 0 *and* both success rates are within τ of 1. The learned model matches the oracle for planning purposes on this task.
- INCONCLUSIVE: CI crosses 0 in a middle-of-the-road regime. The sample size is insufficient to discriminate. Report this verdict and run more episodes.

The default tolerance is $\tau = 0.05$. Crucially, MODEL BOTTLENECK is *not* the default when $\hat{\Delta} > 0$; it requires the AC lower bound to be strictly positive.

4.4 Properties

CPG is bounded in $[-1, 1]$, antisymmetric in the role of the two dynamics, and additive in the following sense: if one decomposes a benchmark suite into disjoint sub-tasks, CPG on the union is the episode-weighted mean of the per-sub-task CPGs. The metric is silent about *why* the model degrades planning (latent shift, prediction-error accumulation, score-function mismatch); follow-up diagnostics are needed to attribute. The metric is, however, sufficient to distinguish model-side failures from planner-side failures at the verdict granularity.

4.5 Robustness to the paired design

Under varied-init sampling the two arms share each episode's initial state, so the design is *paired* and the AC variance above is written for *independent* proportions. Whether that assumption distorts the verdicts is an empirical question about the within-pair correlation ϕ, which we measure directly (Table 1): it is ≈ 0 in the four Reacher cells and only slightly negative on Cartpole (-0.15 and -0.04), so the paired intervals here are essentially interchangeable with AC – Newcombe's half-width is within a few points of AC's in every cell, never tighter – and the independence assumption is neither inflating nor deflating the verdict. We keep AC as the primary report: it is the only estimator whose half-width is a closed form in (n, p_o, p_ℓ) invertible *before* any rollouts (the basis of §5.8), and it is the right tool for the boundary $0/n$ cells where a paired bootstrap degenerates. For the non-degenerate cells we nonetheless corroborate it with two paired-design estimators – Newcombe's paired-difference interval [Newcombe, 1998b] and a paired percentile bootstrap (resampling episode indices jointly) – and a stricter exact McNemar test [McNemar, 1947] with a Holm family-wise correction across cells.

Table 1 reports all three intervals per cell. They agree on whether the interval clears zero in every one of the six non-degenerate cells, so the CI-gated verdicts are *not* artifacts of the independence assumption. The exact McNemar test is more demanding: after the Holm correction the larger-gap MLP cells stay significant, while the smaller-gap TD-MPC2 cells at $n = 30$ – including the LEARNED OUTPERFORMS ORACLE cell – fall short of family-wise significance, which is exactly the regime the power analysis flags for the pooled-150 follow-up (§5.8). A re-analysis script regenerates this table from the committed per-episode data with no GPU.

5 Empirical Study

5.1 Setup

We instantiate the contract on three DeepMind Control Suite tasks [Tassa et al., 2018, Tunyasuvunakool et al., 2020]: Acrobot-swingup (a two-link underactuated pendulum that must build energy and balance upright), Cartpole-swingup (one actuated DoF), and Reacher-easy (a 2-DOF arm reaching a per-episode target – the first two-dimensional action, discretised to a 3×3 torque grid). Acrobot's continuous torque is

Cell ($n = 30$)	$\hat{\Delta}$	ϕ	AC CI	Newcombe CI	paired-boot CI	McNemar p (Holm)
Reacher RS × MLP	+0.300	0.00	[+0.11, +0.45]	[+0.12, +0.48]	[+0.13, +0.47]	0.004 (0.019)
Reacher RS × TD-MPC2	+0.200	0.00	[+0.03, +0.34]	[+0.05, +0.37]	[+0.07, +0.33]	0.031 (0.094)
Reacher CEM × MLP	+0.333	0.00	[+0.14, +0.49]	[+0.15, +0.51]	[+0.17, +0.50]	0.002 (0.012)
Reacher CEM × TD-MPC2	+0.233	0.00	[+0.06, +0.38]	[+0.07, +0.41]	[+0.10, +0.40]	0.016 (0.062)
Cartpole RS × TD-MPC2 (s5)	+0.167	−0.15	[−0.02, +0.34]	[−0.03, +0.36]	[−0.03, +0.37]	0.180 (0.180)
Cartpole CEM × TD-MPC2 (s5)	−0.267	−0.04	[−0.48, −0.02]	[−0.48, −0.01]	[−0.50, −0.03]	0.077 (0.154)

Table 1: The six non-degenerate cells, with the measured within-pair correlation ϕ, under three interval estimators (independent-proportions Agresti–Caffo and two paired-design estimators) plus the exact McNemar test with Holm-adjusted p in parentheses. With $\phi \approx 0$ (Reacher) to slightly negative (Cartpole) the paired intervals are not tighter than AC; the three intervals nonetheless agree on clearing zero in every cell. The stricter Holm–McNemar leaves the small-gap $n = 30$ TD-MPC2 cells short of family-wise significance, reinforcing the pooled-150 follow-up of §5.8. Regenerated from committed per-episode data by `experiments/paired_intervals_audit.py`.

discretised to a five-level set $\{-1, -0.5, 0, +0.5, +1\}$. Success at step t is the DMC dense reward clearing a task threshold. Two planners are used: random-shooting MPC ($N_{\text{cand}} = 50$, horizon $H = 15$, 750 dynamics evaluations per plan call) and a Cross-Entropy Method planner of comparable compute ($3 \times 24 \times 15 = 1{,}080$ evaluations). Episodes run for at most $T = 500$ steps.

Sampling the task distribution. Each episode draws a fresh initial state (and, for Reacher, target), and the two CPG arms share the per-episode env seed by index so the comparison is *paired*: episode k starts from the same state in both arms. We pool three seeds. This is the design the metric's own honesty discipline demands – a single fixed initial state estimates only $P(\text{success} \mid \text{that state})$, not the task. §5.4 shows why it matters.

5.2 Oracle dynamics

We construct the oracle by instantiating a private `dm_control` environment inside a $(\text{state}, \text{action}) \to \text{state}$ callable. Each call reconstructs $(q_{\text{pos}}, q_{\text{vel}})$ from the flat observation via atan2, writes them into the private env's MuJoCo physics, calls `physics.forward`, steps once with the candidate torque, and returns the new flat observation. The construction is side-effect-free from the caller's perspective; the reconstruction is lossy modulo 2π revolutions but Acrobot dynamics depend only on sin / cos of the joint angles. We verify the oracle reproduces `env.step` to numerical precision ($|\Delta| < 10^{-5}$) over a fifty-step random-policy rollout; this regression test is part of the repository's CI.

5.3 Learned dynamics

We train a Markovian MLP (two hidden layers of width 64, ReLU activations) on 2,000 random-policy transitions collected from ten episodes of 200 steps each. The input concatenates the six-dimensional observation with a five-dimensional one-hot action; the output predicts the next observation. Training runs for 200 epochs with Adam (learning rate 10^{-3}), batch size 256, and a 10% held-out validation split at the transition level. The final validation MSE is 0.026. We deliberately choose an MLP rather than a recurrent architecture: Acrobot is fully observed and Markov, so temporal context adds parameters without adding capacity.

5.4 The metric as self-correction: a fixed-initial-state artifact

A first version of this study evaluated every arm at a *single fixed initial state* – the consequence of a deterministic env reset (`task_kwargs={"random": 0}`) and a fresh env per episode, so all episodes, across all seeds, began from the same configuration and only the planner's internal randomness varied. On Acrobot that fixed start happens to be an unusually easy swing-up. The oracle planner solved 30% of episodes under random-shooting MPC and, under CEM, 88% pooled across three seeds, while the learned arm stayed at 0. The gap looked like a textbook MODEL BOTTLENECK: CPG = +0.30 (random-shooting) rising to +0.88 (CEM, AC CI [+0.81, +0.92]).

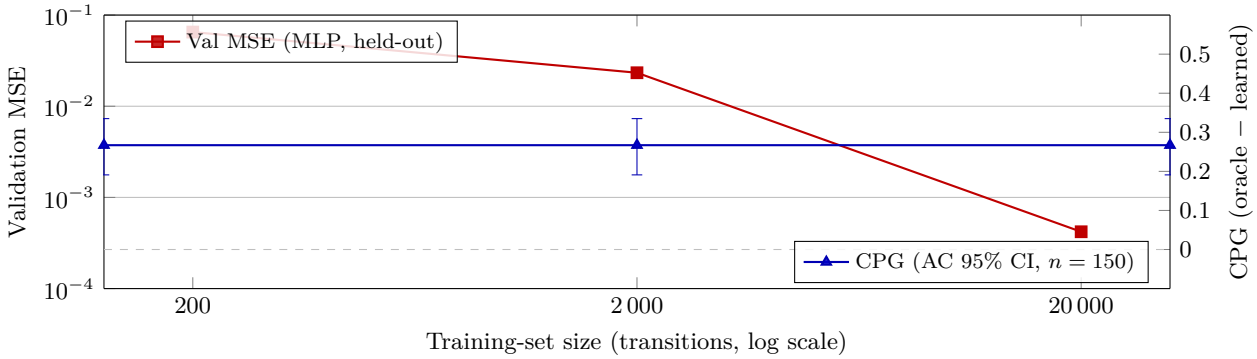

Figure 1: Validation MSE (red, log scale, left axis) drops by ∼150× across the training-set sweep while CPG (blue, right axis) stays exactly flat at +0.267 with the same Agresti–Caffo 95% CI [+0.191, +0.335] in every cell. Predicting better did not plan better. Numbers from `results/dmc_acrobot/cpg_sweep.json`.

Initial state	Planner	Dynamics	Oracle	Learned	CPG (AC 95% CI), verdict
fixed, n=10	random-shoot	MLP	0.30	0.00	+0.30 [−0.06, +0.56], INCONCLUSIVE
fixed, pooled 150	CEM	TD-MPC2	0.88	0.00	+0.88 [+0.81, +0.92], MODEL BOTTLENECK
task, n=10	random-shoot	MLP	0.10	0.10	+0.00 [−0.30, +0.30], INCONCLUSIVE
task, pooled 150	CEM	MLP	0.033	0.020	+0.013 [−0.027, +0.053], PLANNER BOTTLENECK
task, pooled 150	CEM	TD-MPC2	0.033	0.027	+0.007 [−0.035, +0.049], PLANNER BOTTLENECK

Table 2: Acrobot-swingup: the same arms at a fixed initial state vs. sampled over the task's initial-state distribution. The fixed start is an easy swing-up; sampling the task collapses the oracle planner to ∼3% and flips the verdict from MODEL BOTTLENECK to PLANNER BOTTLENECK. Fixed-init numbers are the v0.17 release; task-level from `results/dmc_acrobot/{cpg,cem_cpg_sweep}.json` (`varied_init: true`).

One observation already hinted that the model was not the real story. Sweeping the MLP's training-set size across nearly three orders of magnitude (200 → 20,000 transitions) dropped its held-out validation MSE by ∼150× while leaving the gap, its CI, and the verdict *exactly unchanged* (Figure 1): prediction and decision quality were fully dissociated – the value-(in)equivalence phenomenon that Grimm et al. [2020] and massoud Farahmand et al. [2017] formalise. Better prediction bought no planning success, which is consistent with a model bottleneck but *equally* consistent with a planner that cannot exploit any model from this state.

Sampling the task's initial-state distribution settles it. Drawing a fresh initial state per episode (paired across arms) and pooling three seeds, the oracle's success rate *collapses*: from 0.30 to 0.10 at $n = 10$ under random-shooting, and from 0.88 to 0.033 pooled at $n = 150$ under CEM (Table 2). With the oracle planner itself solving only ∼3% of random starts, the gap closes (CPG = +0.013 and +0.007 for the two learned-dynamics families, both AC intervals straddling zero) and the verdict turns to PLANNER BOTTLENECK: even a perfect model would not help, because the search cannot solve the task from a typical start. The fixed-start MODEL BOTTLENECK was an artifact of an easy initial condition.

This is the strongest single piece of evidence for the metric's purpose: a calibrated, interval-gated statistic, run honestly over the task distribution, overturned a headline that a point estimate at one configuration would have published. The remaining environments are reported at the task level throughout.

5.5 Robustness on Acrobot: published model, stronger planner, perturbation

The Acrobot PLANNER BOTTLENECK is robust to the obvious upgrades. Swapping the bespoke MLP for a published TD-MPC2 world model [Hansen et al., 2024] (both as a latent encoder + dynamics + decoder adapter and as an MLP retrained on TD-MPC2 rollouts) and swapping random-shooting for CEM does not rescue either arm: with the oracle planner itself solving only ∼3% of random starts, every Acrobot cell is INCONCLUSIVE at $n = 10$ and PLANNER BOTTLENECK when pooled. An action-burst perturbation sweep (`DropNextActions`(k), $k \in \{0, 1, 5\}$, fired once per episode) likewise stays INCONCLUSIVE – with both arms

Planner	Dynamics	Oracle	Learned	Raw CPG	AC 95% CI, verdict
Random-shooting	MLP on TD-MPC2 data	0.933	0.000	+0.933	$[+0.76, +0.99]$, MODEL BOTTLENECK
Random-shooting	TD-MPC2	0.933	0.767	+0.167	$[-0.03, +0.34]$, INCONCLUSIVE
CEM	MLP on TD-MPC2 data	0.467	0.000	+0.467	$[+0.25, +0.62]$, MODEL BOTTLENECK
CEM	TD-MPC2	0.467	0.733	−0.267	$[-0.48, -0.02]$, LEARNED OUTPERFORMS ORACLE

Table 3: DMC Cartpole-swingup, TD-MPC2 `model_size = 5`, task-level (three seeds, $n = 30$ pooled). Three verdict branches fire: MODEL BOTTLENECK (MLP arms), INCONCLUSIVE (RS × TD-MPC2), and LEARNED OUTPERFORMS ORACLE (CEM × TD-MPC2). For the last cell the paired bootstrap CI is $[-0.50, -0.03]$ (the 2×2 table is both= 10, oracle-only= 4, learned-only= 12, neither= 4), so both intervals clear zero, though the upper bound is near zero at $n = 30$. Numbers from `results/dmc_cartpole/_size5_pooled.json`.

Planner	Dynamics	Oracle	Learned	Raw CPG	AC 95% CI, verdict
Random-shooting	MLP on TD-MPC2 data	0.933	0.067	+0.867	$[+0.67, +0.96]$, MODEL BOTTLENECK
Random-shooting	TD-MPC2	0.933	0.300	+0.633	$[+0.40, +0.78]$, MODEL BOTTLENECK
CEM	MLP on TD-MPC2 data	0.467	0.067	+0.400	$[+0.18, +0.58]$, MODEL BOTTLENECK
CEM	TD-MPC2	0.467	0.367	+0.100	$[-0.15, +0.34]$, INCONCLUSIVE

Table 4: DMC Cartpole-swingup at `model_size = 1`, task-level (same protocol as Table 3). Three cells are MODEL BOTTLENECK; the CEM × TD-MPC2 cell is INCONCLUSIVE (+0.100, CI crosses zero). Numbers from `results/dmc_cartpole/_pooled.json`.

near zero the perturbation has nothing to separate. The heterogeneity in this paper therefore comes from the other two environments, to which we now turn.

5.6 Cartpole-swingup: model bottlenecks, an inconclusive cell, and a learned model that beats the oracle

We replay the four-arm setup on DMC Cartpole-swingup (one actuated DoF), sampling the task distribution, at two TD-MPC2 capacities (`model_size` $\in \{1, 5\}$, 1×10^6 env steps each). Three seeds pooled, $n = 30$ per arm. The TD-MPC2 checkpoints were trained for this re-run; we therefore read the size-5 and size-1 blocks as two independent task-level snapshots, not as a clean capacity ablation (a fixed-vs-varied or size-vs-size contrast would confound the initial-state change with the separate trainings). Even so, the per-block verdicts are striking: the gate fires *three different branches* on this one environment.

The headline cell is CEM × TD-MPC2 at `model_size = 5`: the learned model lets the CEM planner solve 0.733 of episodes against the oracle planner's 0.467, so CPG = −0.267 and the verdict is LEARNED OUTPERFORMS ORACLE. The AC interval $[-0.48, -0.02]$ and a paired bootstrap $[-0.50, -0.03]$ (the varied-init arms are paired by initial state) both clear zero. We flag two limitations rather than over-sell it: $n = 30$ leaves the upper bound close to zero, and the result rests on a single trained checkpoint; this is the cell most warranting a pooled follow-up. The mechanism is plausible: CEM, a weaker planner here than random-shooting on Cartpole's oracle (0.467 vs 0.933), exploits the smoother learned latent dynamics to find solutions it cannot find against the true dynamics in the same budget. The remaining Cartpole cells are MODEL BOTTLENECK (the MLP-on-TD-MPC2 arms, where the learned dynamics is clearly worse) or INCONCLUSIVE (RS × TD-MPC2 at size 5, +0.167; CEM × TD-MPC2 at size 1, +0.100 – both CIs straddle zero), so a single environment exercises three of the gate's branches.

5.7 Third environment: DMC Reacher-easy

To anchor the MODEL BOTTLENECK pole of the heterogeneity, we replay the four-arm grid on DMC Reacher-easy: a 2-DOF arm reaching a per-episode randomized target (the varied-init protocol genuinely re-randomizes the target each episode). It is the first environment with a *two-dimensional* action (discretised to a $3 \times 3 = 9$ torque grid), and the oracle reconstruction is exact rather than lossy (the observation exposes `qpos` and `qvel` directly; the target is recovered from the finger-to-target vector), verified to reproduce `env.step` to $< 10^{-16}$. TD-MPC2 at `model_size = 1`, 1×10^6 env steps, three seeds pooled to $n = 30$ per arm.

Reacher is the clean MODEL BOTTLENECK case: the oracle solves the reach perfectly (1.000), *both* learned

Planner	Dynamics	Oracle	Learned	Raw CPG	AC 95% CI, verdict
Random-shooting	MLP on TD-MPC2 data	1.000	0.700	+0.300	[+0.11, +0.45], MODEL BOTTLENECK
Random-shooting	TD-MPC2	1.000	0.800	+0.200	[+0.03, +0.34], MODEL BOTTLENECK
CEM	MLP on TD-MPC2 data	1.000	0.667	+0.333	[+0.14, +0.49], MODEL BOTTLENECK
CEM	TD-MPC2	1.000	0.767	+0.233	[+0.06, +0.38], MODEL BOTTLENECK

Table 5: DMC Reacher-easy, task-level (three seeds, $n = 30$ pooled, TD-MPC2 `model_size = 1`). The oracle solves the reach in every cell (1.000); both learned arms are clearly non-zero (0.667–0.800); all four cells are MODEL BOTTLENECK on non-degenerate gaps. Numbers from `results/dmc_reacher/_pooled.json`.

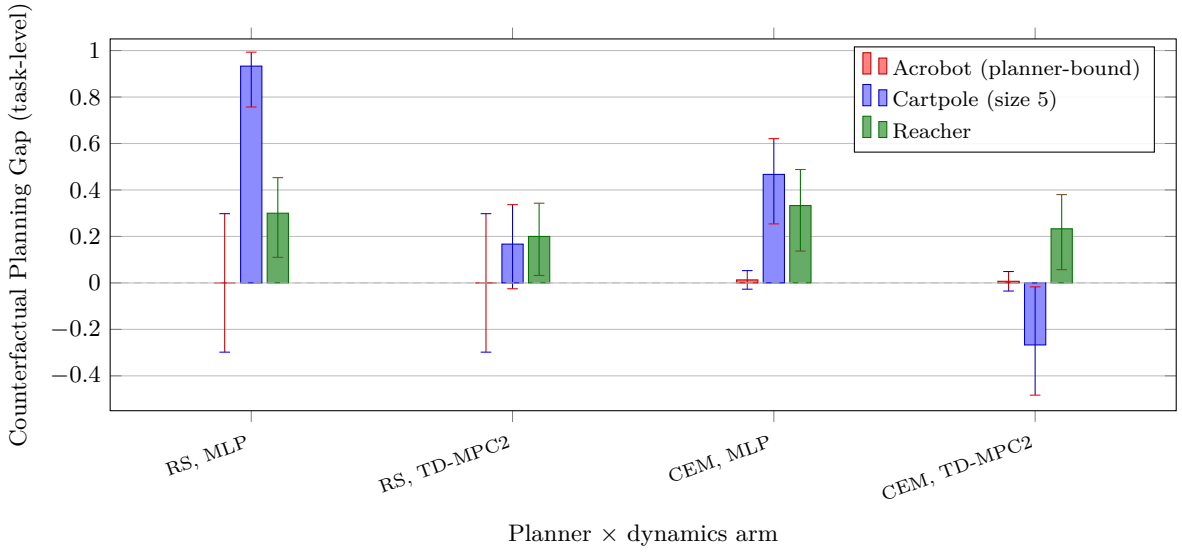

Figure 2: Task-level CPG (varied initial state) with Agresti–Caffo 95% CI error bars, across three environments and four planner-dynamics arms. The verdict is heterogeneous: Acrobot (red) sits at ≈ 0 – the oracle planner barely solves random starts, so the gap is PLANNER BOTTLENECK / INCONCLUSIVE; Reacher (green) is strictly positive MODEL BOTTLENECK in every cell; and Cartpole size 5 (blue) spans the range, from a large MODEL BOTTLENECK (RS × MLP) to a *negative* bar at CEM × TD-MPC2 whose interval clears zero – LEARNED OUTPERFORMS ORACLE. Sample sizes: Acrobot RS cells $n = 10$ and CEM cells pooled $n = 150$; Cartpole and Reacher pooled $n = 30$. The dashed line marks zero gap.

arms are clearly non-zero (0.667–0.800), and yet every cell is MODEL BOTTLENECK – not because a learned arm is pinned at zero, but because the AC lower bound on a genuine, non-degenerate gap (+0.20 to +0.33) is strictly positive. The verdict here tracks gap *magnitude*, not just presence: it ranks the learned dynamics by how much planning success they forfeit relative to a perfect model, while the metric's gate stays well clear of zero. Together the three environments place the gate at three different verdicts – PLANNER BOTTLENECK (Acrobot), a mix dominated by MODEL BOTTLENECK with a LEARNED OUTPERFORMS ORACLE cell (Cartpole), and uniform MODEL BOTTLENECK (Reacher).

Figure 2 aggregates the task-level gaps across the three environments and the four planner-dynamics arms; the bars span from the negative LEARNED OUTPERFORMS ORACLE cell on Cartpole, through the near-zero PLANNER BOTTLENECK / INCONCLUSIVE cells, to the strictly-positive MODEL BOTTLENECK cells on Reacher.

5.8 Power analysis: how many episodes before a ranking is trustworthy

The verdict gate turns from a post-hoc label into a planning tool once it is read forward: given hypothesised success rates and a target precision, how many episodes per arm does the AC interval need? Because the half-width depends only on the rates and n (§4), this is pure arithmetic, computable before any rollout. Table 6 reports the per-arm n needed to reach a half-width of 0.05 (an interval of ±5 percentage points) for a range of oracle rates with the learned arm at 0 – the degenerate regime several cells (e.g. the MLP-dynamics

Oracle rate	n for hw ≤ 0.10	n for hw ≤ 0.05	n for hw ≤ 0.02
0.30	84	327	2021
0.50	98	387	2403
0.70	84	327	2021
0.90	46	153	881

Table 6: Per-arm episode count needed to reach a target Agresti–Caffo half-width (learned arm at 0, $z = 1.96$). Reproduced by `python -m experiments.power_analysis`; numbers from `results/power_analysis.json`.

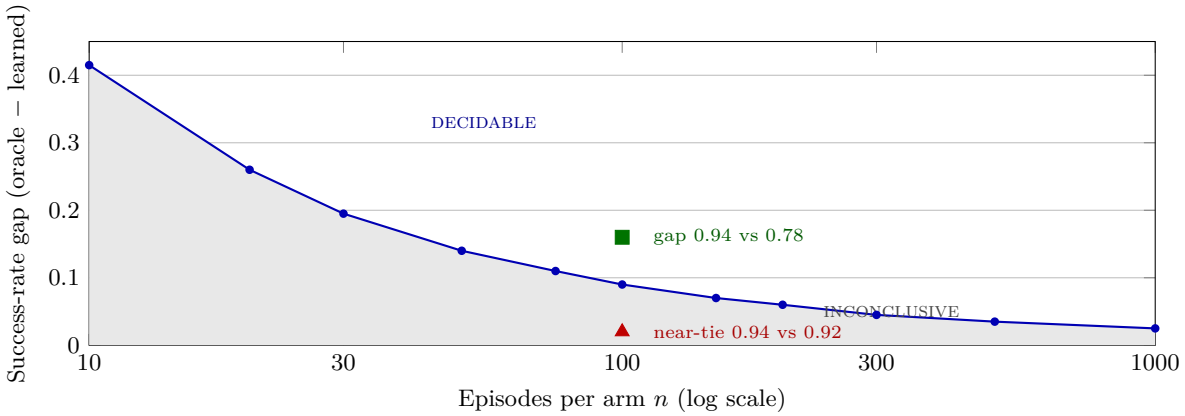

Figure 3: Verdict detectability map at oracle rate 0.94. The curve is the smallest oracle-minus-learned success-rate gap whose Agresti–Caffo 95% CI clears zero at a given per-arm n (computed by `detectable_gap_at_n`). Below it the gate returns INCONCLUSIVE; above it the ranking is decidable. A leaderboard near-tie (0.94 vs 0.92, gap 0.02) reported at $n = 100$ (red triangle) sits inside the INCONCLUSIVE region – the ordering is not supported by its own sample size; a wider gap (0.94 vs 0.78, green square) is decidable at the same n. A point estimate cannot show which side of this boundary a reported ranking sits on.

arms) occupy.

The same arithmetic audits a point-estimate leaderboard. Consider two systems reported at success rates 0.94 and 0.92 over $n = 100$ episodes per arm with no interval – a plausible top-of-table near-tie. The AC interval on that difference has half-width 0.074 and *straddles zero*: the reported ranking gap is statistically indistinguishable from noise at the sample size used, and separating the two to a half-width of 0.05 would require $n = 209$ per arm. A wider gap (0.94 vs 0.78) is decidable at the same $n = 100$ (hw $= 0.095$, interval clears zero); a mid-table near-tie (0.78 vs 0.75) is not. This is the knowledge a leaderboard cannot carry: not which number is larger, but whether the ordering survives its own sample size. A reader holding only point estimates cannot tell a real ranking from a coin flip; the power table makes the distinction mechanical.

A traced decision closes the loop. The Cartpole `model_size = 1`, CEM, TD-MPC2 cell returned INCONCLUSIVE at $n = 30$ (CPG $= +0.100$, half-width 0.24), as did the size-5 LEARNED OUTPERFORMS ORACLE cell whose interval barely clears zero. The table prescribes the per-arm count that would shrink each interval to a target precision and let the gate commit (or recant); reporting that number, rather than silently presenting an under-powered verdict as if it were decisive, is what *decision-grade* is meant to denote.

Figure 3 draws the same fact as a decision boundary: at a fixed oracle rate, the smallest success-rate gap the verdict can commit on shrinks with n, and a reported ranking sits on one side of that boundary or the other. The near-tie at $n = 100$ falls inside the INCONCLUSIVE region; only a wider gap, or a larger n, crosses into the decidable region.

6 Discussion and Limitations

The empirical study spans three DMC control tasks (Acrobot-swingup, Cartpole-swingup, Reacher-easy), two planners (random-shooting and CEM), and two learned-dynamics families (a bespoke MLP and TD-MPC2),

each evaluated over the task's initial-state distribution with the two arms paired by initial state and three seeds pooled. The headline is not a single number but a *pattern*: across the three tasks the gate commits to four of its five branches – PLANNER BOTTLENECK (Acrobot), MODEL BOTTLENECK (Reacher, most Cartpole cells), LEARNED OUTPERFORMS ORACLE (high-capacity Cartpole under CEM), and INCONCLUSIVE (several moderate-n near-ties). The study is deliberately scoped to simulated, fully-observed control where an oracle dynamics is available; on hardware-in-the-loop or physical environments the metric is undefined, and surrogate-oracle variants (e.g. a higher-fidelity model or a domain-randomised reference in the spirit of Tobin et al. [2017]) are future work. The adapter interface (§3) is what makes this breadth cheap: each new cell is one more `dynamics` callable or planner constructor passed into the same `BenchmarkRunner`.

Properties and threats to validity. First, **CPG's magnitude is relative to both the oracle planner and the evaluation distribution, not an absolute property of the model.** The self-correction of §5.4 is the sharpest case: holding the model fixed, the Acrobot gap moves from $+0.88$ to ≈ 0 purely by replacing one fixed initial state with a sample of the task distribution. Read CPG relative to a *stated planner and a stated initial-state distribution*; the verdict gate is more robust than the point estimate because it only asks whether the interval clears zero. Second, **the load-bearing evidence is where the verdict changes – on conditions or on data**, not the degenerate cells where a learned arm sits at $0/n$ and any estimator agrees the oracle wins. The two genuinely informative behaviours are the self-correction (a verdict overturned by sampling the task) and the Cartpole LEARNED OUTPERFORMS ORACLE cell; a gate that flips on evidence and conditions is what a point-estimate leaderboard cannot reproduce. Third, **the design is paired and we report it as such.** Because the two arms share each episode's initial state under varied-init sampling, every non-degenerate cell is corroborated with Newcombe's paired-difference interval and a paired bootstrap alongside the Agresti–Caffo interval, and a Holm-corrected exact McNemar test (§4.5, Table 1); the three intervals agree on clearing zero throughout, while the stricter McNemar test marks the small-gap $n = 30$ TD-MPC2 cells as not-yet family-wise significant – consistent with the pooled-150 follow-up. AC remains the right tool for boundary ($0/n$) cells. Fourth, **the metric requires an oracle dynamics callable and is confined to simulation**; our Acrobot oracle reconstructs $(q_{\text{pos}}, q_{\text{vel}})$ lossily modulo 2π and re-steps MuJoCo, while Reacher's is exact.

Sample size and a confound. The Cartpole and Reacher cells are pooled to $n = 30$; this is thin for the LEARNED OUTPERFORMS ORACLE cell, whose upper bound is near zero, and that cell most warrants a pooled-150 follow-up (the gate doubles as the tool that sizes it, §5.8). The Cartpole `model_size = 5` and `= 1` blocks use *separately trained* checkpoints, so we read them as two task-level snapshots rather than a clean capacity ablation – a size-vs-size or fixed-vs-varied contrast there would confound the comparison with the distinct trainings. The clean fixed-vs-varied ablation is carried by Acrobot (§5.4), where the checkpoint is held fixed.

Other limitations worth flagging explicitly. The discrete-torque action space is a five-level approximation of the continuous control problem; a continuous variant of CEM over a Gaussian per-timestep mean is a straightforward extension. The Acrobot success criterion ($r_t \geq 0.6$) is a binary projection of a dense reward; a continuous variant of CPG that reports the difference of mean returns is straightforward and may be more sample-efficient. The planner score is task-specific and tightly coupled to the environment's reward geometry; a learned score function (predicting the DMC reward directly from the latent state) would remove this coupling, at the cost of a second model component.

7 Conclusion

We have proposed the Counterfactual Planning Gap – the success-rate difference between a fixed planner using oracle versus learned dynamics, reported with an Agresti–Caffo interval, a paired bootstrap where the design is paired, and a five-branch verdict gated on the CI lower bound – and packaged it behind a minimal evaluation contract. Exercised over the task's initial-state distribution on three DMC tasks, the verdict is *heterogeneous*: Acrobot is PLANNER BOTTLENECK (the oracle planner itself solves only $\sim$3% of random starts), Reacher is uniformly MODEL BOTTLENECK on non-degenerate gaps ($+0.20$ to $+0.33$), and high-capacity Cartpole under CEM reaches LEARNED OUTPERFORMS ORACLE (CPG $= -0.27$; the AC and paired-bootstrap intervals both clear zero), with several cells INCONCLUSIVE. The sharpest evidence for the

metric's purpose is a self-correction: an earlier fixed-initial-state evaluation of this same framework reported a large MODEL BOTTLENECK gap on Acrobot, and re-running over the task distribution – the design the metric's own honesty discipline demands – collapsed the oracle and overturned the verdict to PLANNER BOTTLENECK. A calibrated, interval-gated statistic caught a configuration-sensitive artifact that a point estimate would have published. And because the gate is a function of the confidence interval, it doubles as a power tool that sizes a comparison before the rollouts and audits a leaderboard near-tie after them. We argue this kind of calibrated honesty – a metric that refuses to claim more than the data and the evaluation conditions support, and that says so out loud – is the right design target for a methodology that wants to sit usefully next to a fast-moving model literature.

Acknowledgements

This work is independent. It is not affiliated with the AMI (Advanced Machine Intelligence) program at Meta, the LeWorldModel project, the authors of any of the cited papers, or any current or past employer of the author. The repository was developed end-to-end with an LLM coding agent in the loop (Claude Code); a description of the development recipe and the pre-tag adversarial-review pattern that caught most of the metric-correctness bugs along the way is included in the repository at `docs/process/llm_agent_recipe.md`.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Alan Agresti and Brian Caffo. Simple and effective confidence intervals for proportions and differences of proportions result from adding two successes and two failures. *The American Statistician*, 54(4):280–288, 2000.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

Adrien Bardes, Quentin Garrido, Jean Ponce, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. V-JEPA: Latent video prediction for visual representation learning, 2024.

Adrien Bardes et al. V-JEPA 2, 2025. Meta AI technical report.

Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.

Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative interactive environments. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024a.

Jake Bruce et al. Genie 2: A large-scale foundation world model, 2024b. DeepMind technical report.

Pierluca D'Oro, Alberto Maria Metelli, Andrea Tirinzoni, Matteo Papini, and Marcello Restelli. Gradient-aware model-based policy search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Christopher Grimm, André Barreto, Greg Farquhar, David Silver, and Satinder Singh. Proper value equivalence. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

David Ha and Jürgen Schmidhuber. World models, 2018.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023.

Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. MOReL: Model-based offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Yann LeCun. A path towards autonomous machine intelligence, 2022. OpenReview position paper.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026.

Amir massoud Farahmand. Iterative value-aware model learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Amir massoud Farahmand, André Barreto, and Daniel Nikovski. Value-aware loss function for model-based reinforcement learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

Robert G. Newcombe. Interval estimation for the difference between independent proportions: Comparison of eleven methods. *Statistics in Medicine*, 17(8):873–890, 1998a.

Robert G. Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17(22):2635–2650, 1998b.

Seohong Park et al. OGBench: Benchmarking offline goal-conditioned rl. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Anton M. Schäfer, Steffen Udluft, and Hans-Georg Zimmermann. The recurrent control neural network. *Engineering Applications of Artificial Intelligence*, 20(2):197–210, 2007.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind control suite, 2018.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm_control: Software and tasks for continuous control, 2020.

Tingwu Wang, Xuchan Bao, Ignasi Clavera, Jerrick Hoang, Yeming Wen, Eric Langlois, Shunshi Zhang, Guodong Zhang, Pieter Abbeel, and Jimmy Ba. Benchmarking model-based reinforcement learning, 2019.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. MOPO: Model-based offline policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.